instead opt for a *default permit* strategy where requests are permitted to proceed if no ABAC rules match at all. This is the approach you'll take with the Natter API, adding additional ABAC rules that deny some requests and let all others through. In this case the other requests may still be rejected by the existing RBAC permissions enforced earlier in the chapter.

The logic for implementing this default permit with deny overrides strategy is shown in the Decision class in listing 8.10. The `permit` variable is initially set to `true` but any call to the `deny()` method will set it to false. Calls to the `permit()` method are ignored because this is the default unless another rule has called `deny()` already, in which case the deny should take precedence. Open ABACAccessController.java in your editor and add the `Decision` class as an inner class.

Listing 8.10 Implementing decision combining

```
    public static class Decision {
        private boolean permit = true;    #A

        public void deny() {   #B
            permit = false;    #B
        }

        public void permit() {    #C
        }

        boolean isPermitted() {
            return permit;
        }
    }
```